PROG2002 / ASSESSMENT 2 / 01

Good Together

A charity event discovery website for the Gold Coast

Student ID	Last name	First name
24832520	鲁 (Lu)	彧 (Yu)

Introduction/Motivation

Community fundraising depends on people being able to find an activity they can attend and understand the cause it supports. Good Together brings charity event information into one place, with a focus on local participation rather than a complicated purchasing process. A visitor can discover a free shared meal, a fun run or a creative workshop and see how the event contributes to a fundraising goal.

The project uses a fictional Gold Coast organisation and a partner collective. The sample names, contact details, events and fundraising totals are demonstration data. Photographs are illustrative and do not document these events.

Problem Statement

Event details are often scattered across notices and social posts. Visitors need a consistent way to compare dates, locations, categories and ticket prices. They also need to know whether an event is still running, has ended or is unavailable.

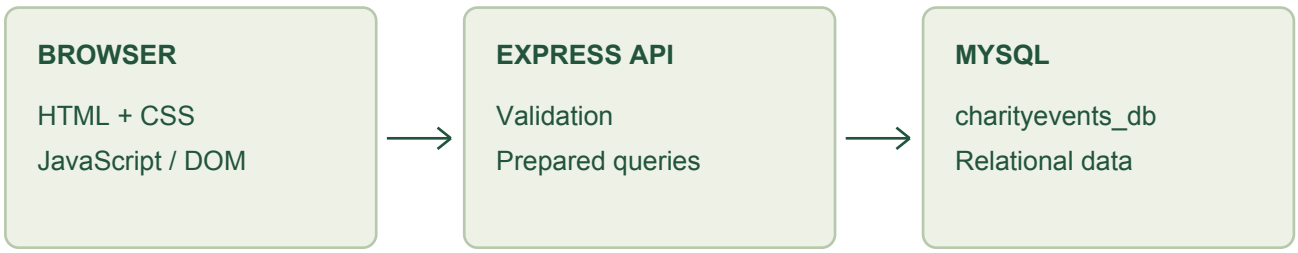
The application must retrieve current data from a server, combine optional search criteria and open the correct event from a result. It must also handle invalid input, missing records and connection failures without leaving the visitor with an unexplained blank page.

Scope: Assessment 2 provides event discovery and a registration placeholder. Booking records, payments and administration are reserved for Assessment 3.

PROG2002 / ASSESSMENT 2 / 02

Solution

The implementation separates presentation, request handling and storage. Express serves the client files and the API from the same origin. The browser never connects directly to MySQL and does not receive database credentials.



Request: GET + query parameters | Response: JSON -> DOM update

From a search to an event card

1. The search form collects an event start date, a suburb and a category. JavaScript prevents a page submission and builds a query string with URLSearchParams.

2. The shared getJSON function calls fetch and awaits the JSON response. Express validates the query, builds the selected conditions and passes their values separately to mysql2.execute.

3. MySQL joins the event, category and organisation tables. The API returns matching records in chronological order, with a count and the event time zone.

4. The client clones an HTML card template, assigns record values with textContent and inserts a document fragment into the results area. No page reload is needed.

State and failure handling

A new search cancels the previous request with AbortController. This prevents a slow response from replacing newer results. A request timeout, loading message, empty state and retry action make the result of each interaction visible. The search URL preserves the criteria, and browser Back restores the previous form values and results.

Register displays “This feature is currently under construction.” Ticket quantities change only the displayed total; no booking or payment is made.

PROG2002 / ASSESSMENT 2 / 03

Web UX

The visitor journey is Home -> Find an event -> Event details. The same navigation appears on every page, including error pages. The home page explains the mission before presenting the event list. Category shortcuts lead to a pre-filtered search.

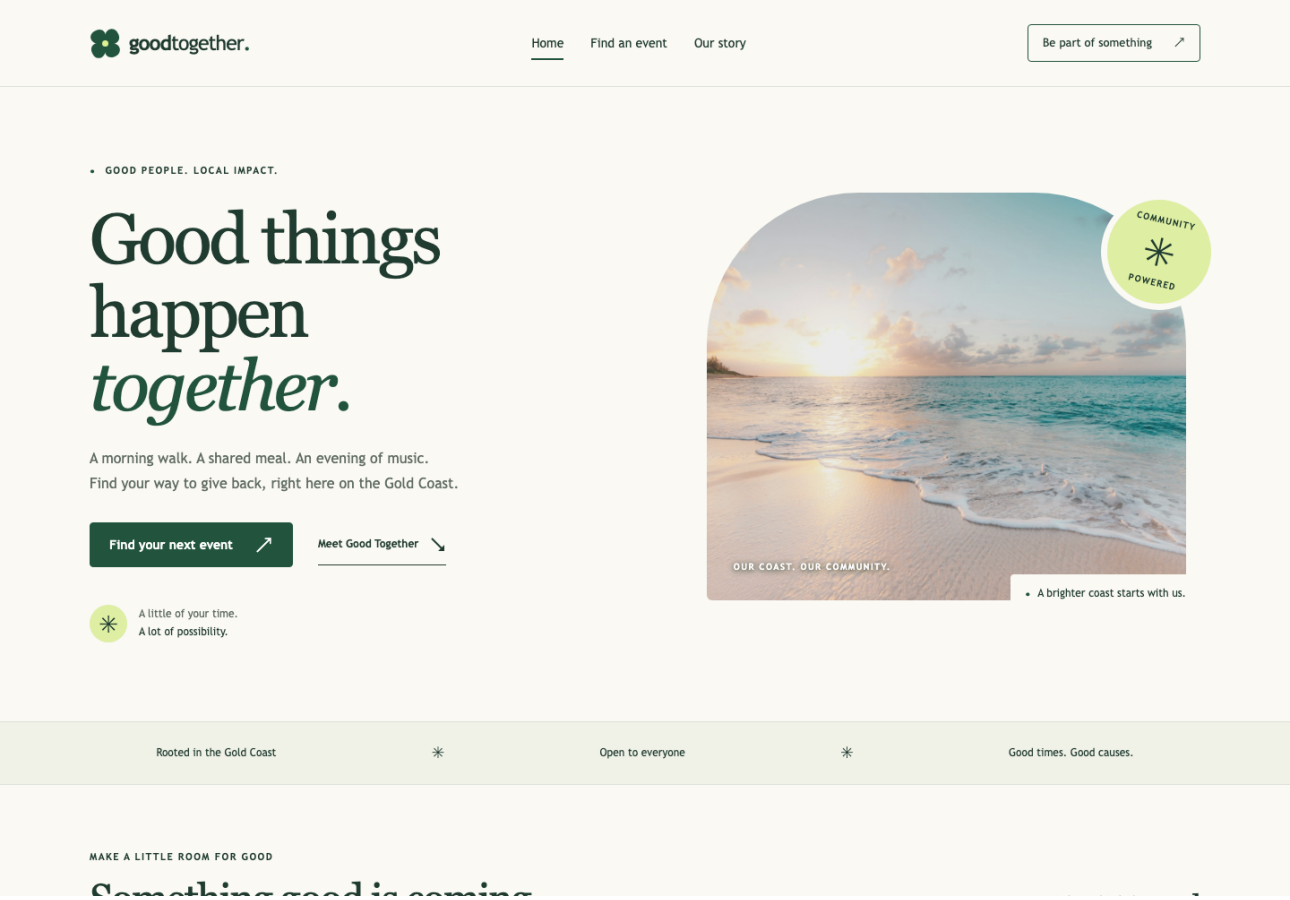


Figure 1. Home page at 1440 px. The organisation introduction and primary event action establish the purpose of the site.

The interface uses a warm neutral background, dark green text and restrained lime accents. Dates, category labels, free entry and suburb names make cards easy to scan. A native date input matches the date filter; a text field supports partial suburb names; and a dropdown shows categories supplied by the API.

At narrow widths, the layout stacks into readable sections without horizontal scrolling. Controls have explicit labels, keyboard focus is visible, and the main content has a skip link. Search status uses an aria-live region, errors use an alert role, and the native dialog supports Escape and restores focus to Register.

PROG2002 / ASSESSMENT 2 / 04

Data Schema



One organisation hosts many events. One category classifies many events.

Entity	Key data and responsibility
organisations	Name, mission and contact details. One organisation may host several events.
categories	A unique name and description for each type of event. One category may classify several events.
events	Foreign keys; event information, times and location; ticket price, fundraising figures, image, accessibility and publication status.

Primary keys identify records. Foreign keys prevent orphaned events, and ON DELETE RESTRICT protects referenced records. Shared details are stored once to reduce duplication.

Amounts use DECIMAL. A zero price means free entry, the fundraising goal must be positive, and the raised amount can exceed the goal. Both insert and update triggers reject invalid date order and negative amounts. Triggers are used because MySQL 5.7 does not enforce CHECK constraints.

Publication status is active or suspended. Current, upcoming and past are derived from start and end times, using UTC plus ten hours. The public listing excludes suspended and ended events; a past event detail remains readable but registration is hidden. An end time equal to the current time counts as past.

The import provides 12 events, 6 categories and 2 organisations. Dates are relative to import day, then remain fixed. Indexes support visibility, date and category filters. All times use Brisbane wall time.

PROG2002 / ASSESSMENT 2 / 05

API design

Method and resource	Purpose
GET /api/events	All current or upcoming active events. Optional date, location and category query parameters are combined with AND.
GET /api/categories	Category IDs, names and descriptions used to populate the filter dropdown.
GET /api/events/:id	One active event with complete details, including an ended event. A suspended or absent record returns 404.
GET /api/health	Confirms that the server can query its database.

A filtered request

Example: GET /api/events?date=2026-09-26&location=Southport&category=1. The date matches the event start date in Gold Coast time. The location is a case-insensitive literal substring. The category is a positive integer ID. GET has no request body; the values are carried in the URL.

The response is JSON with a data array and metadata. An empty match returns HTTP 200 with data: [] and count: 0. The following is an abbreviated response; each returned event also contains the detailed fields used by the client.

```
{
  "data": [{
    "id": 1, "name": "Steps for a brighter coast",
    "category": "Fun run", "location": "Southport",
    "starts_at": "2026-09-26 07:30:00",
    "ticket_price": 25, "event_status": "upcoming"
  }],
  "meta": { "count": 1, "timeZone": "Australia/Brisbane",
    "filters": { "date": "2026-09-26",
      "location": "Southport", "category": 1 } }
}
```

PROG2002 / ASSESSMENT 2 / 06

API behaviour

GET is used for every application resource because Assessment 2 only reads information. POST, PUT and DELETE are intentionally unavailable and return 405 with an Allow header. A registration button does not imply that a booking endpoint exists.

Dates are checked for both format and calendar validity. IDs must be positive whole numbers within the database integer range. Duplicate and unknown filters are rejected. Location length is limited, SQL wildcard characters are escaped, and values are bound as prepared-statement parameters.

Invalid input returns 400; an absent or suspended event returns 404; and an unexpected database failure returns a generic 503. API responses are not cached so time-derived status stays current. The server applies a same-origin content security policy, and the browser renders database text as text rather than HTML.

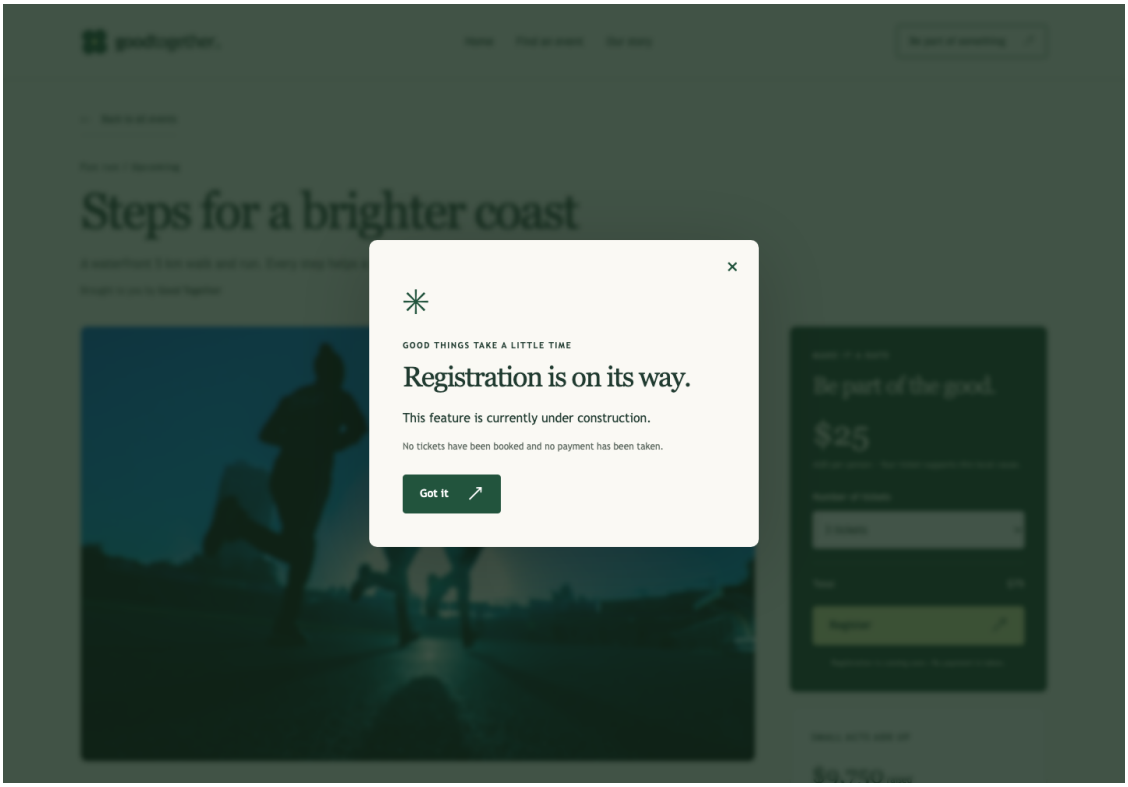


Figure 2. The native registration dialog shows the required placeholder. No booking or payment request is made.

PROG2002 / ASSESSMENT 2 / 07

Verification

Recorded checks

Final verification on 21 September 2026 used Node.js 20.20.0 and MySQL 5.7.28. All 32 backend tests passed. The browser suite passed 32 checks across Chromium and WebKit. These results describe the supplied sample dataset and tested environments; they do not establish compatibility with every browser.

Area	Evidence
Data and API	12 records; valid joins; combined filters; empty results; free tickets; suspended and missing records; 400/404/405/503 responses.
Integrity and safety	Foreign-key rejection; insert/update validation; literal SQL-injection strings; escaped wildcard searches; exact date boundaries.
Browser journey	Home to search to details; clear filters; browser Back; empty-state recovery; error retry; ticket totals; modal and keyboard focus.
Responsive layout	Home, Search and Details at 390, 768 and 1440 px, with no horizontal overflow or broken loaded images.

Backend tests create and remove a uniquely named disposable MySQL database. They do not overwrite the demonstration database. A WebKit focus-restoration issue was found during testing and corrected with an explicit dialog close handler.

PROG2002 / ASSESSMENT 2 / 08

Running and reviewing the project

Local installation

Extract the two source archives into the same directory so api and clientside are siblings. Import api/database/charityevents_db.sql into a fresh MySQL installation, copy api/.env.example to api/.env and enter the local credentials. The SQL file does not drop existing tables.

```
cd api
npm ci
npm run db:check
npm start
# Open http://127.0.0.1:3088
npm test
```

The Node.js connection file is api/event_db.js. The main routes are in api/app.js, the browser rendering helpers are in clientside/js/common.js, and page-specific behaviour is in home.js, search.js and event.js. The client files and images are served locally, so the interface does not depend on a CDN.

Repository and remaining submission actions

Private repository: github.com/364962/prog2002-a2-charity-events. The milestones are database setup (17 September), APIs and validation (18 September), client pages (19 September), and final verification and documentation (21 September). This report, the browser checks and the completed README form the fourth milestone. Commit dates record the actual submission of each stage.

The student still needs to grant the marker repository access, record or replace the narration to demonstrate personal understanding, upload the final video to SCU OneDrive, and submit the required files and links through Blackboard. Source archive names currently use student ID 24832520; replace that prefix if the required university username differs.

Preparation disclosure

Generative AI tools assisted with code implementation, testing and report preparation.

PROG2002 / ASSESSMENT 2 / 09

Interface evidence and references

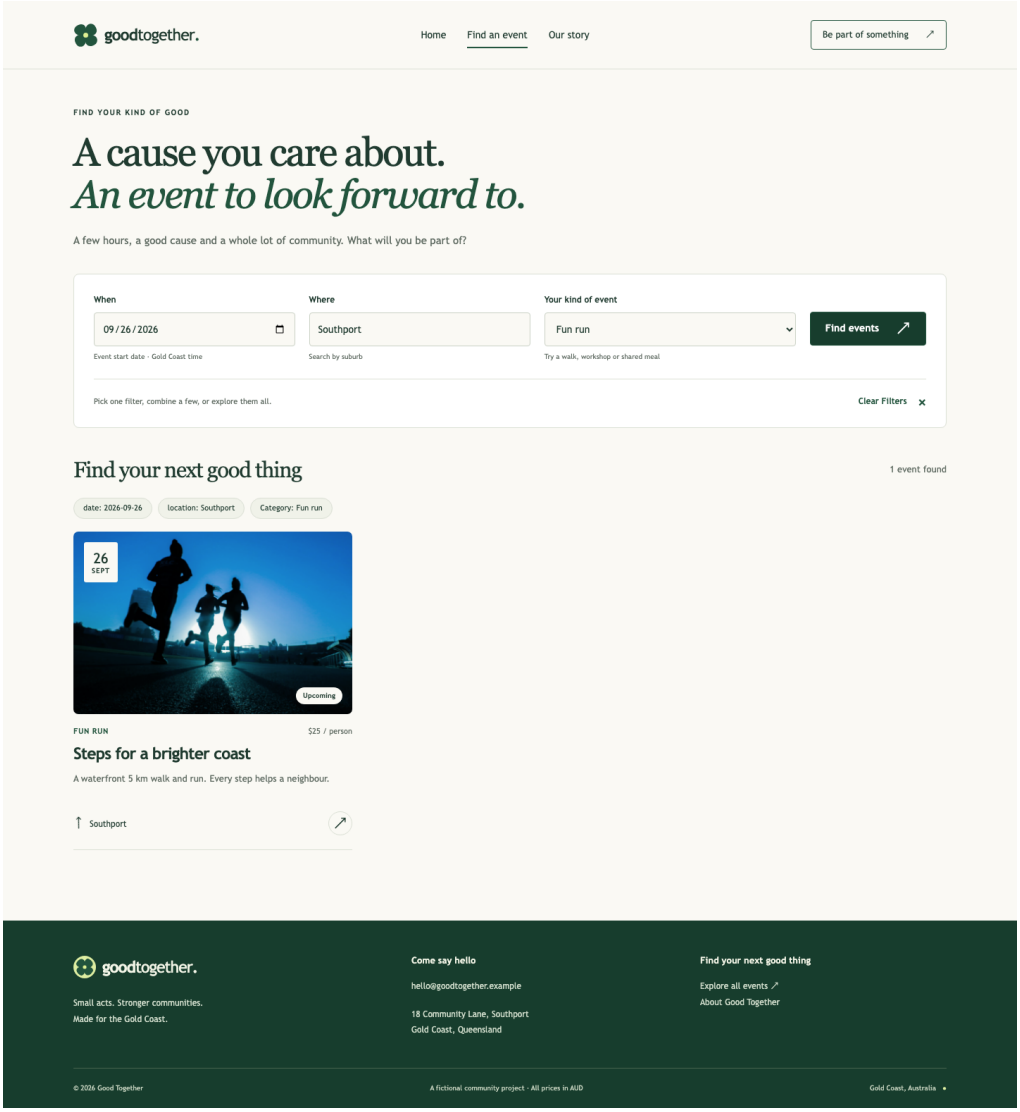


Figure 3. The combined search returns the matching event and displays the selected filters. This is a capture of the running application connected to MySQL.

Implementation references

Express: error handling. expressjs.com/en/guide/error-handling.

MySQL2: promise API, connection pools and prepared statements. sidorares.github.io/node-mysql2/docs.

Illustrative image sources and license links are listed in clientside/assets/CREDITS.md. The supplied Assessment Brief and Report template informed the required scope and report headings.